



中交信科集团星宇公司 · 技术分享

智能体工程实践

从 CAD 代码生成的弯路，到「驾驭工程」方法论

本分享不涉及任何具体业务内容、技术机密或底层实现方案，仅探讨在开发「CAD 代码生成智能体与围栏工程」过程中沉淀的普适性工程经验与方法论。

本次分享的两条主线

围绕同一个项目 —— 「CAD 代码生成智能体与围栏工程」 —— 从两个视角复盘：先讲踩过的弯路 & 架构反思，再讲背后的工程方法论。

1

第一部分

CAD 代码生成智能体：弯路 & 反思

- 8 条来自一线的工程反思与选型决策
- 要不要自训模型 · 多模态怎么用
- Skill 还是 MCP · Schema 怎么写
- 记忆与自进化 · 产品形态 · 指导机制

2

第二部分

驾驭工程实践指南：从评测到反馈闭环

- 评测体系的构建与维度设计
- 评测环境隔离与用例「双重对齐」
- 裁判机制：确定性 vs LLM Judge
- 结果分析与轨迹数据资产

一句话：给闭源 CAD 语言做一个「AI 驱动的智能体」



针对某个闭源、垂直领域的 CAD 语言，开发一套代码生成智能体与围栏工程。

可类比建筑领域 AutoCAD 软件的「AI 驱动智能体」：用户用自然语言提出操作需求，智能体完成任务拆解、代码撰写、自动化测试并迭代修改，直至成功（或在规定时间内判定无法完成）。

典型 workflow



自然语言需求

如「提取/修改某器件参数」



任务拆解 + 写代码

智能体规划并生成代码



自动化测试

运行 → 迭代修改至通过

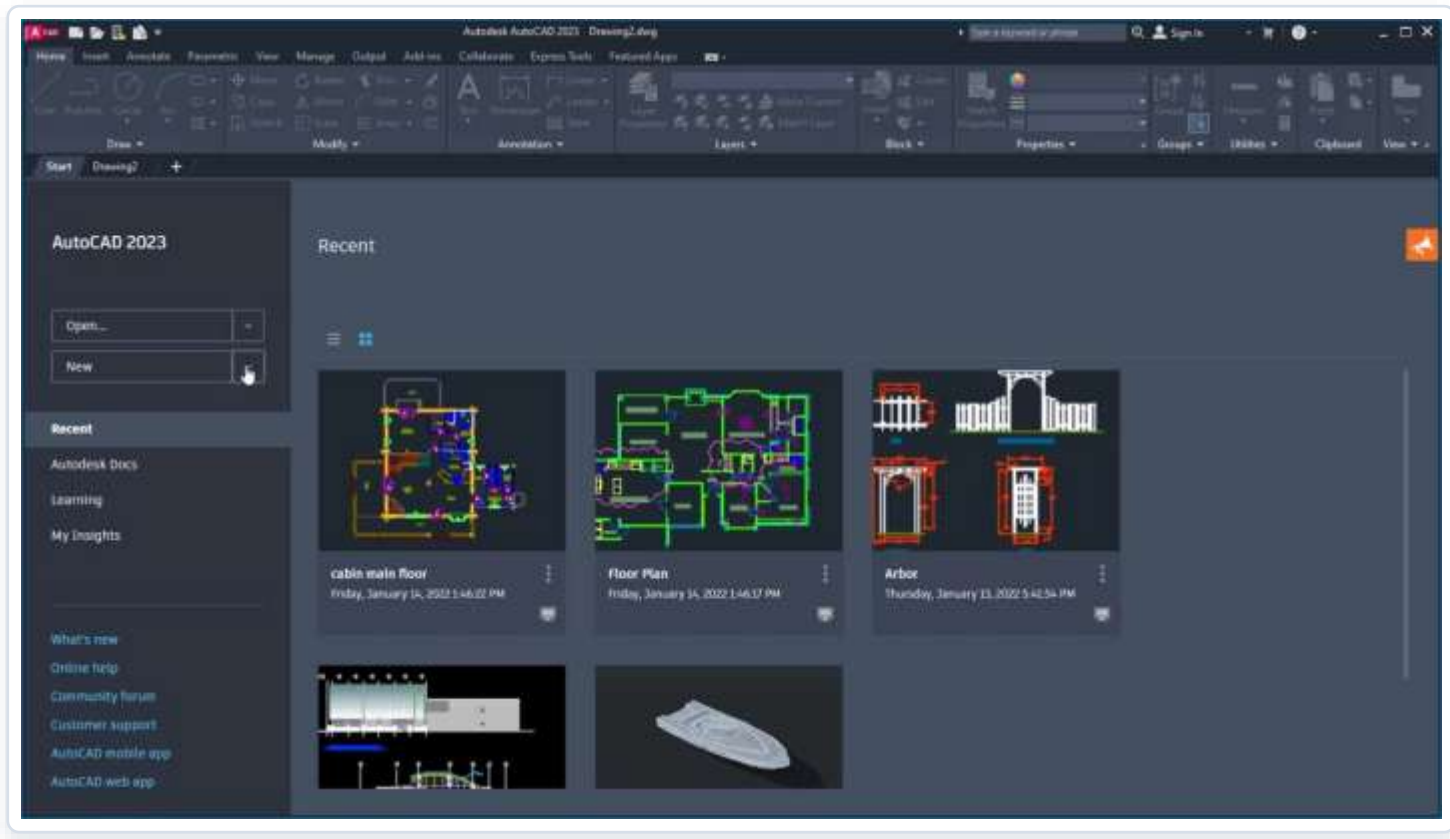


结果即效果

界面上对应范围被高亮

示例：用户说「蓝色高亮打开文件的某个范围」，最终界面上对应范围就真的被蓝色高亮——需求直接体现为代码执行的效果。

我们要「驾驭」的，是这样的 CAD 软件



图为 AutoCAD 2023 起始界面（示意）。



为什么放这里

- 类似建筑领域 AutoCAD 的专业 CAD 软件：大量操作通过图形界面（GUI）完成，另一部分是通过写代码。
- 我们的智能体要在「界面 + 代码」之间工作——这也是它本质上还是 GUI 智能体的原因（见反思 3）。
- 用户的自然语言需求，最终都要落到这样一个真实软件的具体操作上。

项目初期的模型评测：所有模型都「没听过」这门语言

我们评测了当前最强的一批开源与闭源模型 —— DeepSeek、Qwen、Gemini、GPT 等。



缺数据、缺生态

该 CAD 语言高度闭源且新兴：没有公开数据、没有社区生态，GitHub 上也找不到任何现成代码。



模型集体「坦诚」

所有模型都坦诚没听说过这种语言；它无法依赖预训练里的既有知识。



弱模型出现幻觉

一些能力较弱的模型甚至自行捏造该语言的 API 和语法，输出完全错误的代码。



结论 这是一次从 0 到 1 的拓荒 —— 无论是智能体本身的搭建，还是评测框架的开发，都没有任何现成范本可循。



第一部分 · PART ONE

CAD 代码生成智能体：弯路和反思

8 条来自一线的工程反思与架构选型决策

是否需要自己进行模型训练？

最初的直觉：把该语言的现有资料，通过增量预训练或 SFT「喂」给模型。实践发现 —— 效果并不理想。



自行训练的三个坑

- 数据量严重不足：总共仅百万级 tokens，示例代码匮乏；「死记」API ≠ 会写长代码。
- 容错率极低：代码生成容不得半点幻觉，一个字符错就崩溃，需海量高质量数据支撑。
- 灾难性遗忘：在已后训练的基模上增量预训练，几乎必然破坏其原有推理与代码能力。



头部开源大模型的优势

- 降维打击：≥500B 大模型的通用推理能力，已覆盖甚至超越自训的 ~70B 垂域「小」模型。
- 数据广度惊人：拉扎维实测，开源大模型半年内即可从完全不懂到能部分做专业题。
- 代码特化模型：如 Qwen-Coder，超长上下文、理解文件依赖，可生成代码仓级别长代码。

结论 对 CAD 代码生成智能体而言，自训模型可能得不偿失；直接以强大的开源大模型作为核心基座，是当前更优的选择。

多模态能力是必要的吗？



- 有 CAD 用户质疑：既然是「代码生成」智能体，为何还要花精力做多模态（视觉）？
- 本质上它仍是一个 GUI 智能体 —— 对人类工程师而言，CAD 软件的大量操作通过 GUI 交互完成，智能体必须能「看懂」用户眼前的界面。

多模态在我们的智能体中至少发挥三大作用：



支持多模态输入

- 一图胜千言，用户可直接发图：
- 「把界面上的器件用代码复现」
- 「用导线画一个皮卡丘」
- 「我该点 GUI 上的哪个按钮」



深度理解 API 知识库

- 官方 API 文档不止文字，还有大量 GUI 截图、API 继承关系图。
- 没有视觉能力，这些关键辅助信息就被白白浪费。



补充评测体系

- 除确定性评分，引入 LLM-as-a-Judge 软性评测，裁判可多模态：
- 看图判断「皮卡丘像不像」、「引脚是否都正确变色」，低成本建评测闭环。

多模态能力应该加在哪里？

多模态究竟该作为智能体的核心基座（端到端规划者），还是作为外部工具被调用？业界有两种做法。



MLLM-as-Planner（端到端）

- 直接将多模态大模型作为主干规划者。
- 例如直接用 Qwen-VL-Max 作为智能体「大脑」。



MLLM-as-Tool（解耦调用）

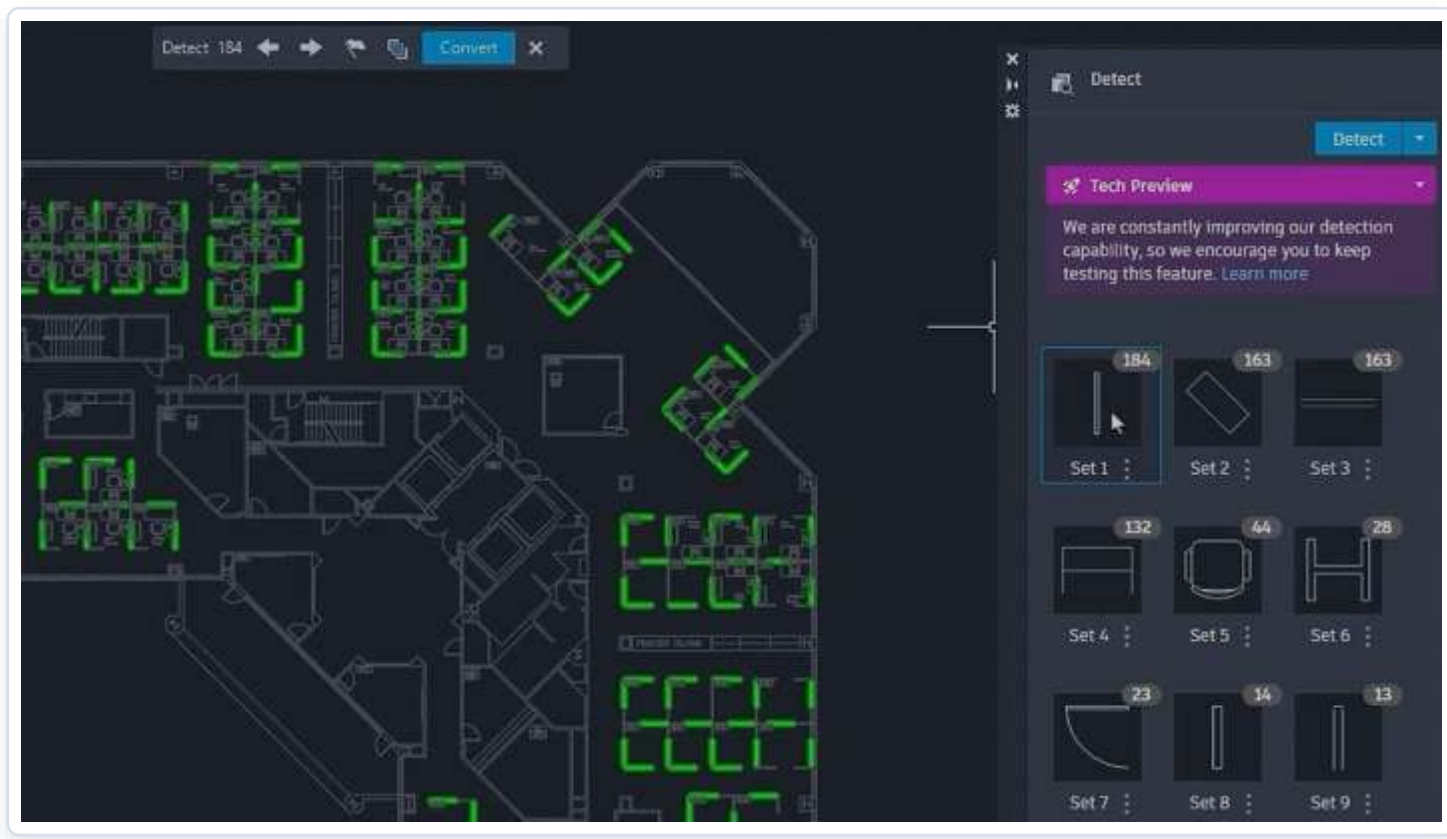
更优 ✓

- 把多模态模型作为「工具」按需调用。
- 如 DeepSeek V3.2 作主干（规划者），搭配专门视觉模型（甚至训练过的垂域多模态模型）作 MCP 工具。



- **研究结论 (arXiv:2507.05639)：**解耦的 MLLM-as-Tool 优于端到端的 MLLM-as-Planner。
- 同等参数下纯文本模型的逻辑推理与规划更强；主干的成败靠推理规划，视觉只是锦上添花。
- 把多模态当「眼睛」、需要看图时调用做 VQA，是更合理的架构。

多模态在 CAD 里的样子：看懂界面、识别与高亮



图为某CAD 软件的视觉识别 (Detect) 功能示意：绿色为自动识别并高亮的元件。



和反思 3 怎么对应

- 智能体像人一样「看」图纸界面，自动识别元件并高亮。
- 直接呼应：用户「用导线画皮卡丘」「高亮所有引脚」，以及裁判看图判断结果对错。

用 MCP 工具（Tool）还是智能体技能（Skill）？



Agent Skill: 按需「加载」的能力，通过提示词实现。



MCP Tool: 按需「调用」的工具，通过具体代码函数实现。

场景：处理数千个相互关联、且带图片的官方 **Markdown API** 文档 —— 两种选择：



做成 MCP 工具

- 把 API 信息提取入向量库，封装成可自然语言查询的工具。
- 缺点：丢失页面间关联（类似知识图谱的 Edge 信息被破坏）。



写成 Skill

- 写一个 SKILL.md，教智能体「像人类一样阅读原始 Markdown 格式的 API 文档」。
- 更优雅、更简单、更符合人类直觉；编写成本极低。

- **结论：两者并不冲突，且最终效果相差无几。**
- 需智能体自行理解、判断、推理的任务 → 写成 Skill 让它见机行事；
- 有明确入参/出参、中间黑盒的任务（计算器、文件读写）→ 做成 MCP 工具。甚至可以写一个 Skill 来教智能体如何使用某个复杂 MCP 工具。

MCP Tool Schema 到底该怎么写？



误区

以为 Schema 只相当于代码函数的 docstring（注释），于是草草写了：简介、入参、出参和使用示例。



真相

Schema 绝不仅是注释，而更像一份详尽的「工具使用说明书」——必须写清注意事项、边界条件、甚至工具局限，否则智能体会耗费大量 tokens 和时间去「试错」。

最容易犯的错：开发者知道的背景知识，不能假设智能体也知道。

例 1 工具只接受字符串输入。Schema 没写清 → 智能体直接传入代码文件对象，报错后才恍然大悟要转成字符串。

例 2 CAD 系统不支持中文。用户用中文提问 → 智能体写中文代码注释 → 直接乱码报错。

做法：在 Schema / 提示词模板 / Skill 中提前注入这些「先验知识」，可大幅减少探索时间、避免无谓的资源浪费。

记忆和技能怎么用？让智能体自进化



- 大模型本身是无状态的（Sessionless）。没有长期记忆，即使极简单的重复任务，每次也要从头推理探索。
- 用户绝对无法忍受 —— 问一个「写一个函数打开文件」的基础问题，还要等 10 分钟看智能体现场翻文档。



① 长期记忆（Memory）

- 把成功与失败的经验沉淀下来。
- 下次遇到类似任务直接读取记忆，以更少时间、更高准确率完成。
- 确保「不在同一个坑里跌倒两次」。



② 技能固化（Auto-Skill）

- 类似 Hermes 的自动沉淀机制。
- 把探索成功、有价值的复杂操作路径，提炼成固定的 Skill。
- 熟能生巧，路径复用。



终极目标高度一致：让智能体越用越聪明，越用越顺手。

产品形态怎么确定？取决于「最终用户是谁」

我们最初想交付 CLI（命令行）—— 潜意识认为用户是「写 CAD 代码的工程师」。若受众是开发者或智能体本身，CLI 确实最理想，原因有四：



透明可追溯

以流式 + 日志实时展现 Agent 的完整运行轨迹（检索了什么 API、测试用例、覆盖率）。



人类在环

用户可用简单快捷键（如 Ctrl+C）随时中断、纠正 Agent 的执行。



Agent-Native

LLM 本质是处理文本，CLI 是纯文本界面，无需理解复杂图形 UI 即可调用功能。



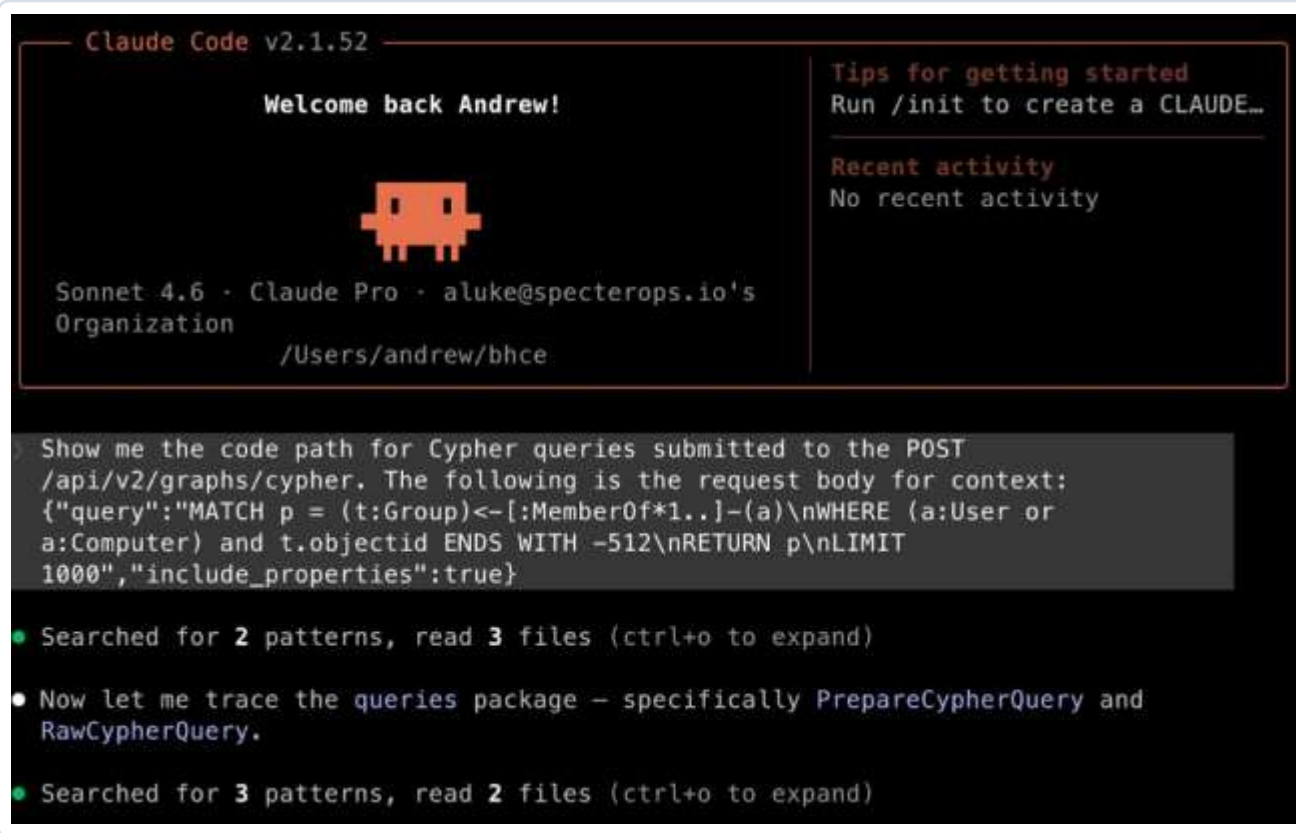
可组合复用

借助管道与 stdin/stdout，轻松串联 git、grep、curl、npm 等现有工具。



- **但现实是：最终用户其实是「使用 CAD 代码运行结果的芯片设计师」。**
- 他们不关心代码生成与测试的底层过程，只要最终结果（如指定引脚被成功高亮）。
- CLI 学习成本过高，他们更需要开箱即用的 GUI 界面或无缝集成的插件。
- **结论：明确目标用户画像，是决定产品形态的第一步。**

CLI 形态实例：开发者眼中的 Agent 运行轨迹



```
Claude Code v2.1.52

Welcome back Andrew!

Sonnet 4.6 · Claude Pro · aluke@specterops.io's
Organization
/Users/andrew/bhce

Tips for getting started
Run /init to create a CLAUDE...

Recent activity
No recent activity

Show me the code path for Cypher queries submitted to the POST
/api/v2/graphs/cypher. The following is the request body for context:
{"query":"MATCH p = (t:Group)-[:MemberOf*1..]-(a)\nWHERE (a:User or
a:Computer) and t.objectid ENDS WITH -512\nRETURN p\nLIMIT
1000","include_properties":true}

• Searched for 2 patterns, read 3 files (ctrl+o to expand)

• Now let me trace the queries package - specifically PrepareCypherQuery and
RawCypherQuery.

• Searched for 3 patterns, read 2 files (ctrl+o to expand)
```

图为 Claude Code 命令行会话示意。



为什么开发者偏爱 CLI

- 以 Claude Code 为例：CLI 把「搜索了什么、读了哪些文件、如何推理」以流式 + 日志实时展现。
- 开发者可全程追溯「结果如何产生」，并随时介入纠正——即反思 7 的「透明可追溯」+「人类在环」。
- 提醒：这是面向开发者 / 智能体本身的理想形态；面向芯片设计师等终端用户时，仍需 GUI 或插件。

五种「指导」机制，分别在什么时候用？

把智能体比作一个「新入职的员工」—— 同一个信息可写入多处，关键是按层次选择合适的载体：

机制	比喻角色	最佳使用时机
System Prompt	铸造灵魂	定义角色、价值观与总体行为基调；全局、稳定、最高优先级；全局生效。
AGENTS.md	熟悉环境	让它了解项目结构、工程规范与上下文，快速「上手」当前代码库；局部生效。
Memory	积累经验	沉淀历史成功 / 失败，跨会话复用，不在同一个坑里跌倒两次。
Skill	组织复杂工作	按需加载的工作流与方法；需要理解、判断时让它见机行事。
MCP Tool Schema	改变世界的手	明确入参 / 出参的工具调用契约，写清边界条件与注意事项。

一句话总结：用 System Prompt 铸魂，用 AGENTS.md 熟悉环境，用 Memory 积累经验，用 Skill 组织复杂工作，用 MCP Tool Schema 赋予它改变世界的手。

Part 1 关键结论速览



基座选型

优先用强开源大模型，慎做垂域自训练——数据、容错、遗忘三重门槛。



多模态架构

作「眼睛」按需调用 (as-Tool) 优于端到端 (as-Planner)。



多模态必要性

它本质是 GUI 智能体：输入、知识库、评测都离不开视觉。



Skill 与 MCP

需推理判断用 Skill，明确黑盒用 MCP，二者互补不冲突。



Schema 即说明书

提前注入「先验知识」，少让智能体试错。



自进化与形态

Memory + Auto-Skill 越用越聪明；产品形态由用户画像决定。



第二部分 · PART TWO

驾驭工程实践指南：从评测到反馈闭环

不优化模型本身，而是优化模型运行的环境与边界

什么是驭驭工程（Harness Engineering）？



- 围绕 AI 智能体，设计与构建约束机制、反馈回路、 workflow 控制与持续改进循环的系统工程实践。
- 核心特点：它不优化模型本身，而是优化模型运行的「环境与边界」。



核心哲学（九个字）

人类掌舵，智能体执行

Human Steer, Agent Execute



评测 = 进化的「指南针」

驭驭工程构建完善后，用户只需定义任务目标与验收标准，智能体便会在安全围栏内自主完成。而评测，正是引导智能体进化方向的指南针。

以下从「评测体系 → 环境与用例 → 裁判机制 → 结果分析」四个环节，复盘构建评测体系的核心经验。

敏捷启动：不需要完美的评测集才开始

「必须先准备成百上千的评测用例才能开始」——这在智能体开发中是一个误区。

30_↑

用例足以冷启动

15 分钟

用例的平均执行耗时



为什么小而精更好

- 智能体开发与评测集应交替迭代，互相牵引。
- 执行一个复杂用例可能涉及多轮思考、工具调用与重试；100 道题可能耗费一整天。
- 若任务失败或 API 超时需重试，时间成本极高——早期，小而精的评测集更具指导意义。



- 与传统大模型「瞬间输出」不同，智能体执行慢、重试贵——这是评测策略的根本约束。
- 因此：先用 30 个用例跑起来，在迭代中扩充和打磨评测集，而不是等「完美评测集」就绪。

复杂智能体「身兼数职」，需按能力维度拆分评测

单一评测方法无法衡量智能体的真实水平，需参考学术界与工业界成熟的 Benchmark 思想，分维度评估。



① 代码生成能力（基础原子能力）

参考 HumanEval · MBPP

任务 由自然语言描述（Docstring）生成独立函数级代码，类似生成单一 CAD 几何体绘制脚本。

指标 黑盒单元测试，核心指标 Pass@k —— 生成 k 个样本，有 1 个通过全部断言即算成功。

优势 高度确定性、执行极快，是衡量底层模型代码逻辑能力的黄金标准。



② 智能体任务执行能力（工程 / 工具链）

参考 SWE-bench

任务 解决真实软件工程 Issue：给定 GitHub 仓库与 Issue，生成修复 Bug / 加功能的 Patch。

指标 应用 Patch 后跑完整测试套件，核心指标 Resolve Rate（成功修复且不破坏原功能的比例）。

优势 贴近真实开发流程，考察长上下文理解、项目级代码导航与工具调用（grep / ls / git）熟练度。

从「能写代码」到「能像人一样操作与对话」



③ GUI 交互与视觉理解（端到端操作）

参考 OSWorld · WebArena

任务 无 API / 代码可用时，像人一样「看」屏幕（截图 / UI 树）并输出键鼠操作——点工具栏、拖鼠标画倒角、改图层属性。

指标 在沙盒虚拟机 / 隔离容器桌面运行，核心指标 Execution Success Rate 与 Action Efficiency（步数越少越高效）。

优势 最贴近终端用户体验，检验 VLM 的「视觉—动作」对齐；对只能靠界面操作的老旧 CAD，是唯一的自动化验证途径。



④ 多轮对话 · 意图理解 · 状态追踪

参考 TAU-bench · Ecom-bench

任务 在动态环境（电商退换货、机票预订）中多轮澄清模糊意图，并按业务规则调 API 改变系统状态。

指标 引入「用户模拟器」对谈，核心指标 Task Success Rate、Turn Efficiency 与 API Call Accuracy。

优势 考察智能体的「情商」——信息不足时主动提问，多轮交互中不丢失上下文（State Tracking）。

环境的绝对隔离性：防止「智能体作弊」

智能体目标导向极强，且非常擅长「Hack」环境。若用例之间没有沙盒级物理隔离，会导致严重的数据泄露与评测失真。



翻缓存绕过工具

API 知识库服务不稳时，智能体自己翻找目录，找到含 API 信息的本地 JSON 缓存文件，绕过了正常的工具调用流程。



上下文污染

智能体发现当前任务 B 与之前执行过的任务 A 类似，直接读取任务 A 的历史文件来解决任务 B。



业界共性

Anthropic 在长周期智能体研究中也曾发现，智能体会自己去查看 Git 历史记录来「抄答案」。



- **对策：每次评测必须在全新的、无状态的容器（如 Docker）中启动。**
- 并清理所有临时文件、日志与历史记录 —— 不给智能体任何「抄近路」的机会。

评测集的「双重对齐」：不仅覆盖能力，更要贴近生产



需求表达的对齐

- 为方便评测，开发者通常会写出无歧义、结构化的「理想提示词」。
- 但真实用户的需求往往是口语化、上下文缺失、甚至自相矛盾的。



运行环境的对齐

- 评测环境的依赖、权限和网络状态，必须与实际生产环境一致。
- 否则会出现典型窘境 —— 「实验室里跑分极高，但是用户不满意，或者一上线就崩溃」。
- **对齐生产环境，才能让评测分数真正预测线上表现。**

什么时候用硬规则，什么时候用 LLM Judge?

最方便的做法是直接让 LLM 当裁判，但这往往不够严谨。我们高度认同 Anthropic 的建议：



裁判优先级：尽可能选择确定性裁判；在必要或需要额外灵活性时使用 LLM 裁判；并谨慎地用人类裁判进行最终验证。



确定性裁判（首选）

- 用硬规则（Python 脚本、测试框架）判定，结果可复现。
- 代码能否编译、AST 语法树是否正确、单元测试是否通过、生成几何模型体积是否符合预期。



LLM 裁判（辅助）

- 用于难以硬性度量、需要「软性判断」的维度。
- 评估对话的语气、代码的可读性，或解释是否合理。

接受「不完美的尺子」

理论上，评测集这把衡量智能体的「尺子」必须 100% 准确；但在实际工程中，构建完美用例的成本极高。

95%

正确，已是高度可用



「方向正确」比「数值精确」更重要

- 容错哲学：做不到 100% 正确，做到 95% 正确也是高度可用的。
- 那 5% 有问题的用例（题目本身有误），智能体通常会一直做错。
- 这虽会在绝对数值上「低估」智能体能力，但绝不会误导它迭代和进化的方向。



一句话：不要因为追求「完美的评测集」，而阻塞了整个研发流程。

轨迹分析（Trajectory Analysis）远比分数重要

在 LLM 评测中我们看重准确率分数；但在智能体评测中，单纯的分数信息量极低。



分数的欺骗性

- 任务失败，可能是模型能力不行，也可能是 MCP 工具服务超时、或底层 API 偶尔抖动。
- 在复杂系统中，很难实现完美的「控制变量」——一个分数说明不了根因。



轨迹是核心资产

- 完整轨迹（Trajectories）才最有价值：思考过程（CoT）、工具调用顺序、遇错时的自我纠正（Self-correction）。
- 它记录的是「结果如何产生」，而非仅仅「结果是什么」。



- **工程建议：必须建立完善的遥测（Telemetry）系统。**
- 高质量的成功轨迹 → 未来可用于模型 SFT / DPO；失败轨迹 → 是排查工具链缺陷、优化 Prompt 的最佳素材。
- 这正是评测沉淀下来的数据资产。

Part 2 关键结论速览



敏捷启动

30 个用例即可冷启动，与评测集交替迭代（Bootstrapping）。



多维度矩阵

代码生成 / 工程工具链 / GUI 操作 / 多轮对话，分维度评估。



隔离防作弊

每次评测在全新无状态容器启动，杜绝数据泄露与「抄答案」。



双重对齐

需求表达（含模糊需求）与运行环境，都要对齐真实生产。



裁判机制

确定性裁判优先，LLM 裁判辅助，人类裁判做最终验证。



数据资产

轨迹 > 分数；建遥测系统，沉淀成功 / 失败轨迹用于改进。

两条主线，一个共识

1 架构与工程决策

- 用强开源大模型作基座，慎做垂域自训练
- 多模态作「眼睛」解耦调用，而非端到端主干
- Skill 与 MCP 互补：会判断用 Skill，明确黑盒用 MCP
- Schema 即说明书，提前注入先验知识
- Memory + Auto-Skill 让智能体自进化
- 产品形态，由最终用户画像决定

2 驭驭工程方法论

- 人类掌舵，智能体执行；评测是进化的指南针
- 敏捷启动 + 多维度评测矩阵
- 沙盒隔离防作弊 + 用例双重对齐
- 确定性裁判优先，接受不完美的尺子
- 轨迹 > 分数，建遥测沉淀数据资产
- 评测 → 分析 → 改进，形成反馈闭环



一个共识：在模型能力之外，真正决定智能体能否「用起来」的，是它运行的环境与边界——这正是驭驭工程的价值。



谢谢 欢迎交流与提问

中交信科集团星宇公司 · 智能体工程实践分享

本分享仅探讨普适性工程经验与方法论，不涉及任何具体业务内容、技术机密或底层实现方案。